

Homework 5 Tools Reliability and a Safe Local Agent

GitHub: <https://github.com/vedantvartakwork/data260-3539>

Final submitted PDF tag hw5 commit: fcf24af64451a922c46316925edd387ce09a5101

Item	Value	Item	Value
SID4	3539	PORT_BASE	8839
PREFIX	s3539	SEED	3539
VERIFY_SEED	263539	DOMAIN_ID	3
Domain	Grocery supply and recall notices	Local model	qwen3:8b
Hardware	Apple M4 MacBook Air	Memory / CPU	16 GB / 10 cores
Platform	macOS 15.3	Ports	API 8839 / UI 5173

This cumulative submission extends the authenticated FastAPI/MySQL application with a manufacturer relationship, Redux Toolkit and Axios, two MCP servers, strict tool schemas, bounded retries, and a local Ollama tool-calling loop with deterministic safety enforcement.

Submission contents

The repository includes RUN_LOG.txt, METRICS.md, REFLECTION.md, TOOL_CONTRACTS.md, AI_USE.md, verification.json, 150 raw fault-injection rows, MCP/agent/API raw outputs, Postman collection and environment files, and this screenshot-backed report.

1 Completion and verification

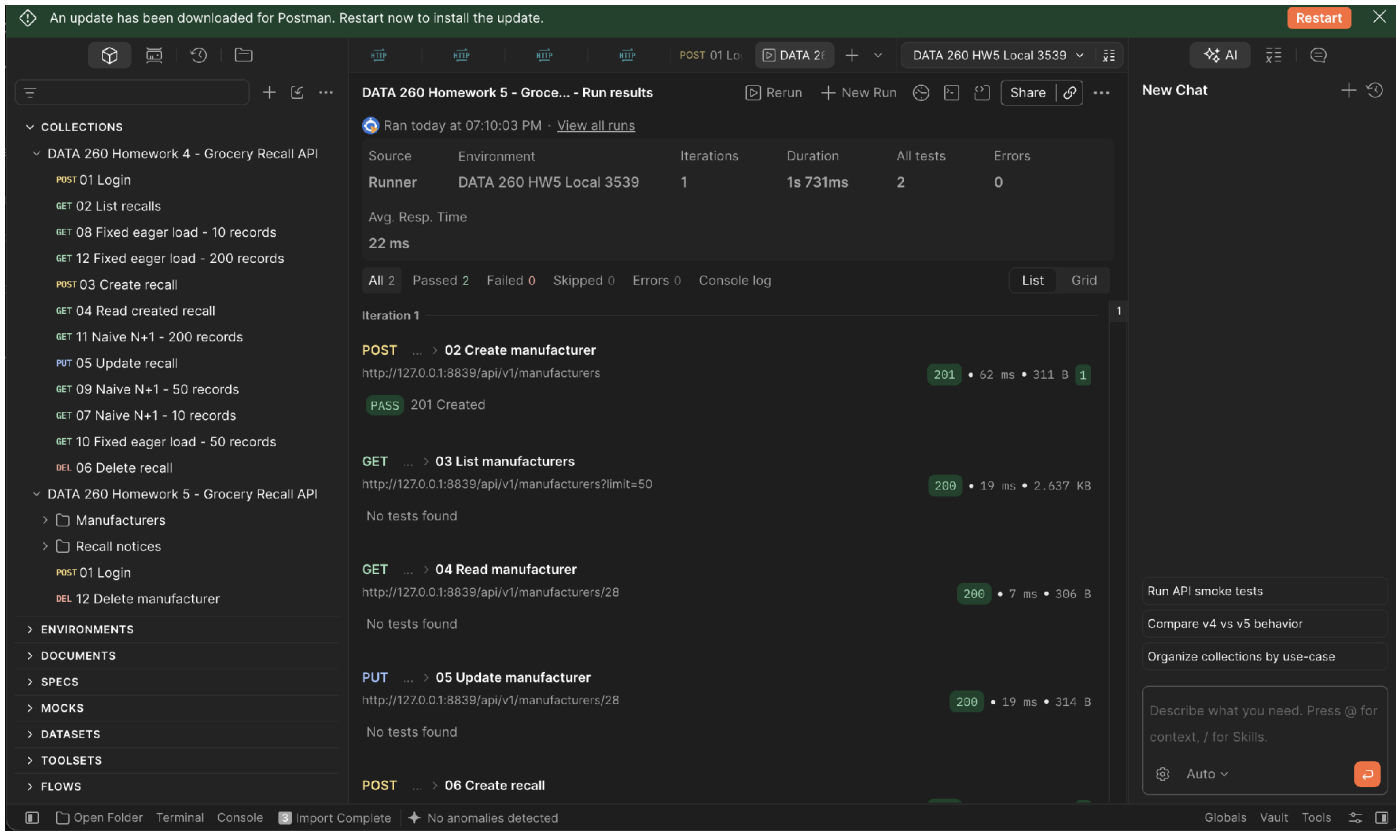
Automated local checks are summarized below; repository access is verified separately.

Area	Evidence	Result
Database/API	12 manufacturers; 500 recalls; 200 events; 14/14 authenticated checks	PASS
React/Redux	Home, create, update, delete; 10-page access to all 500 records	PASS
MealDB MCP	Exactly four live tools demonstrated	PASS
Domain MCP	Exactly search/detail/aggregate; valid and invalid calls	PASS
Reliability	150 deterministic calls; 3-attempt bounded retry	PASS
Offline tests	9/9 tool and safety tests	PASS
Cumulative suite	37/37 Python tests; React production build	PASS
Local agent	Four qwen3:8b scenarios; one enforced safety block	PASS
Verifier	14/14 checks including real STDIO and tag alignment	PASS

The verifier independently inventories both MCP servers, calls one tool from each, checks the live API health endpoint, counts all 150 experiment rows, runs the offline runner and 37-test suite, and rebuilds the React client. Both MCP calls use newly launched STDIO processes; application source is compared to the hw5 tag.

Part 1 Database API actual Postman operations

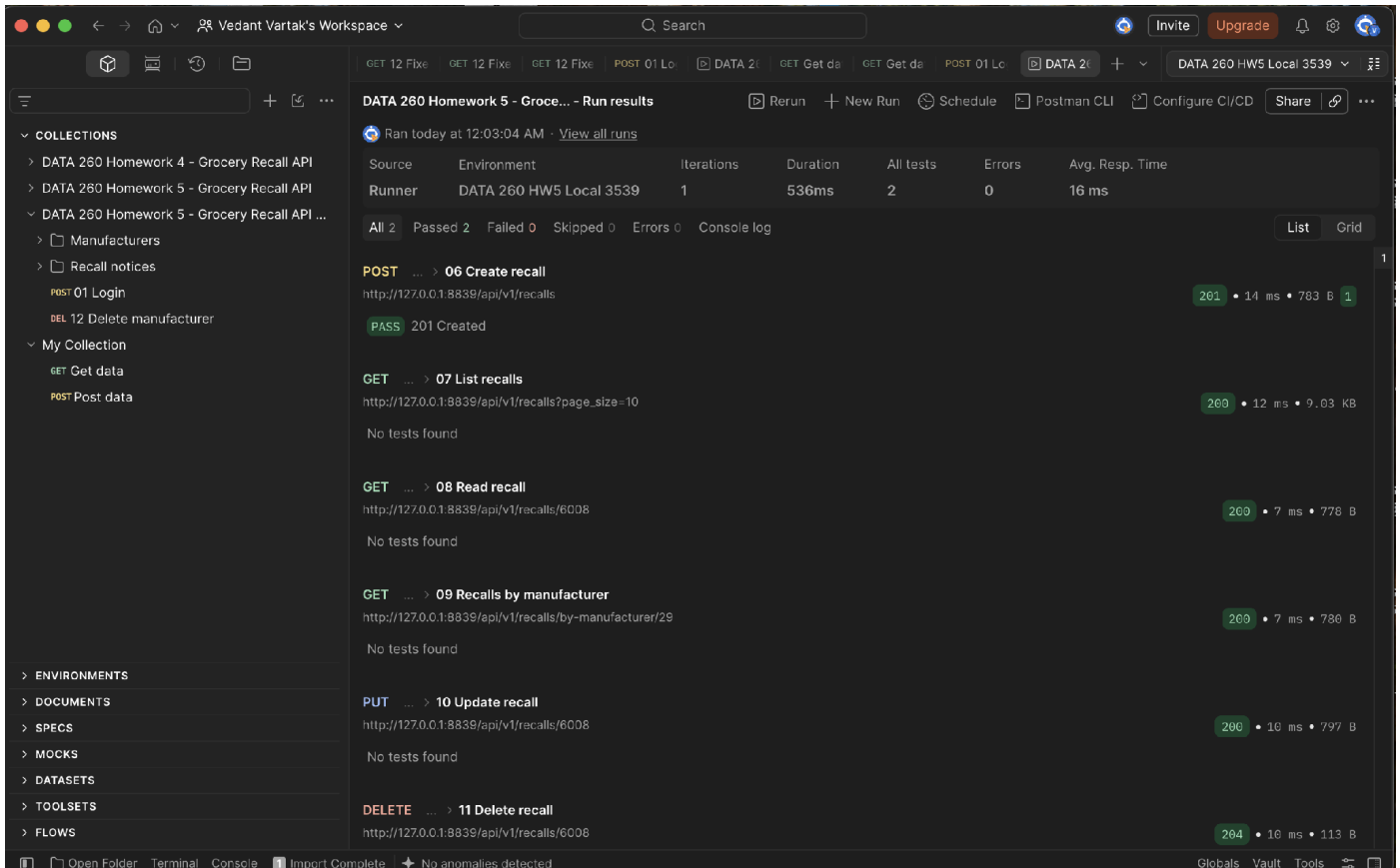
This is the original Postman application capture, not a reconstructed panel. It shows the selected HW5 environment and successful manufacturer operations. It is an earlier temporary demonstration run; the linked recall workflow in Sections 3-6 uses manufacturer 29 and recall 6008 consistently.



Postman Runner UI: HW5 collection, local environment, response statuses, timings, and test summary.

3 Actual Postman Runner evidence recall operations

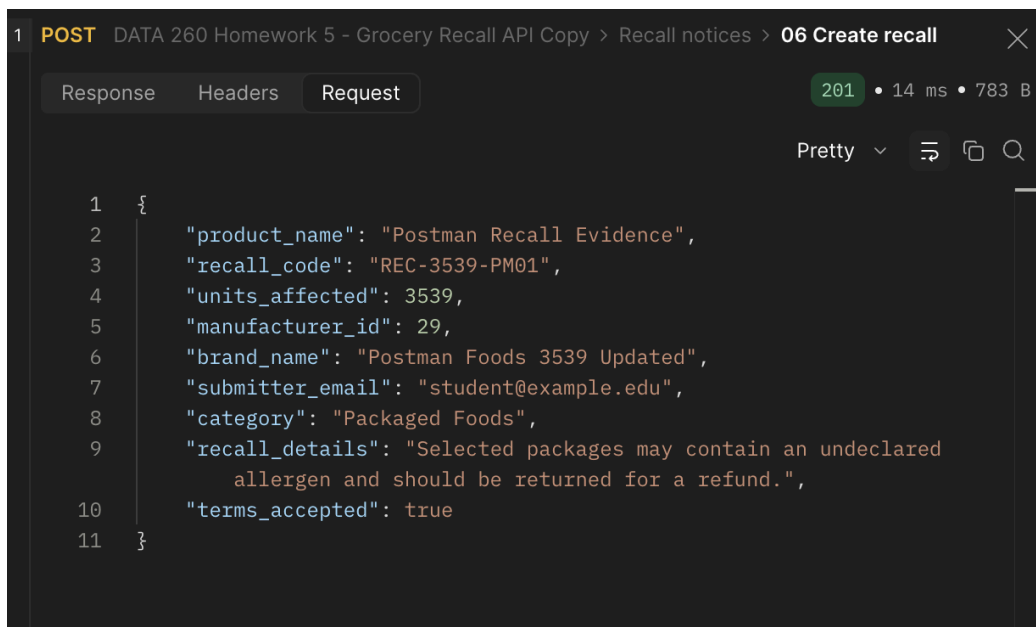
This new Postman run visibly shows recall creation, listing, reading, the manufacturer relationship endpoint, update, and delete. Login was sent before the run. It is a separate run from the manufacturer capture and companion panels: recall ID 6008 and manufacturer ID 29 identify this capture.



Actual Runner results: create 201; list, read, relationship, and update 200; delete 204. Two assertions passed, with zero failed assertions and zero request errors.

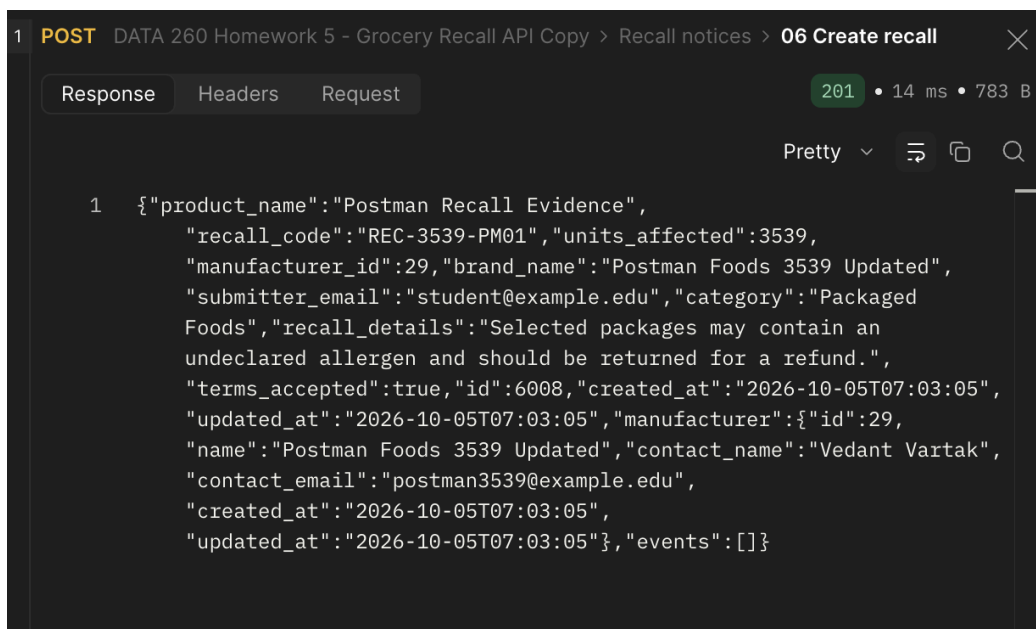
4 Actual Postman request and response recall creation

These are close crops of original Postman application captures from the same run shown above. The first records the submitted JSON request and the second shows the stored 201 response for recall 6008. Original captures are retained.

A screenshot of the Postman application interface. The top bar shows the request details: '1 POST DATA 260 Homework 5 - Grocery Recall API Copy > Recall notices > 06 Create recall'. Below the bar, there are tabs for 'Response', 'Headers', and 'Request', with 'Request' being the active tab. To the right of the tabs, it shows the status '201' in a green box, followed by '14 ms' and '783 B'. Below this, there are icons for 'Pretty' (a dropdown arrow), a refresh icon, a copy icon, and a search icon. The main area displays a JSON request body with line numbers 1 through 11. The JSON is formatted with indentation and line wrapping.

```
1  {
2    "product_name": "Postman Recall Evidence",
3    "recall_code": "REC-3539-PM01",
4    "units_affected": 3539,
5    "manufacturer_id": 29,
6    "brand_name": "Postman Foods 3539 Updated",
7    "submitter_email": "student@example.edu",
8    "category": "Packaged Foods",
9    "recall_details": "Selected packages may contain an undeclared
10                       allergen and should be returned for a refund.",
11    "terms_accepted": true
}
```

Cropped request pane: submitted JSON and 201 Created status; original capture is retained.

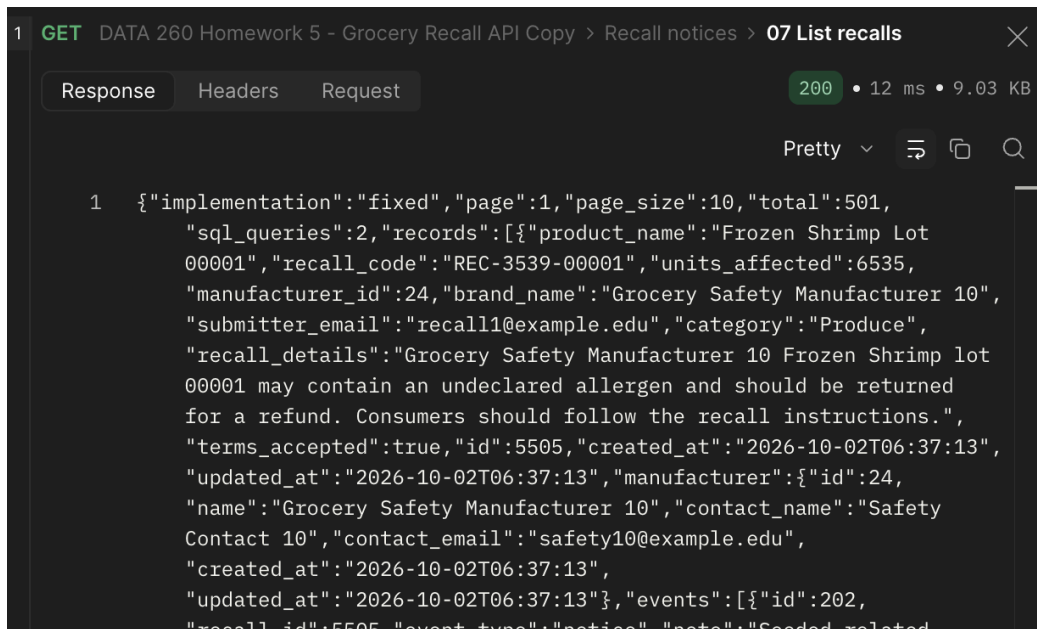
A screenshot of the Postman application interface, showing the 'Response' pane. The top bar is identical to the previous screenshot: '1 POST DATA 260 Homework 5 - Grocery Recall API Copy > Recall notices > 06 Create recall'. The 'Response' tab is now active. The status '201' is still shown in a green box, along with '14 ms' and '783 B'. The main area displays a JSON response body with line number 1. The JSON is formatted with indentation and line wrapping, showing details about the created recall, including its ID (6008), creation and update timestamps, and manufacturer information.

```
1  {"product_name":"Postman Recall Evidence",
    "recall_code":"REC-3539-PM01","units_affected":3539,
    "manufacturer_id":29,"brand_name":"Postman Foods 3539 Updated",
    "submitter_email":"student@example.edu","category":"Packaged
    Foods","recall_details":"Selected packages may contain an
    undeclared allergen and should be returned for a refund.",
    "terms_accepted":true,"id":6008,"created_at":"2026-10-05T07:03:05",
    "updated_at":"2026-10-05T07:03:05","manufacturer":{"id":29,
    "name":"Postman Foods 3539 Updated","contact_name":"Vedant Vartak",
    "contact_email":"postman3539@example.edu",
    "created_at":"2026-10-05T07:03:05",
    "updated_at":"2026-10-05T07:03:05"},"events":[]}
```

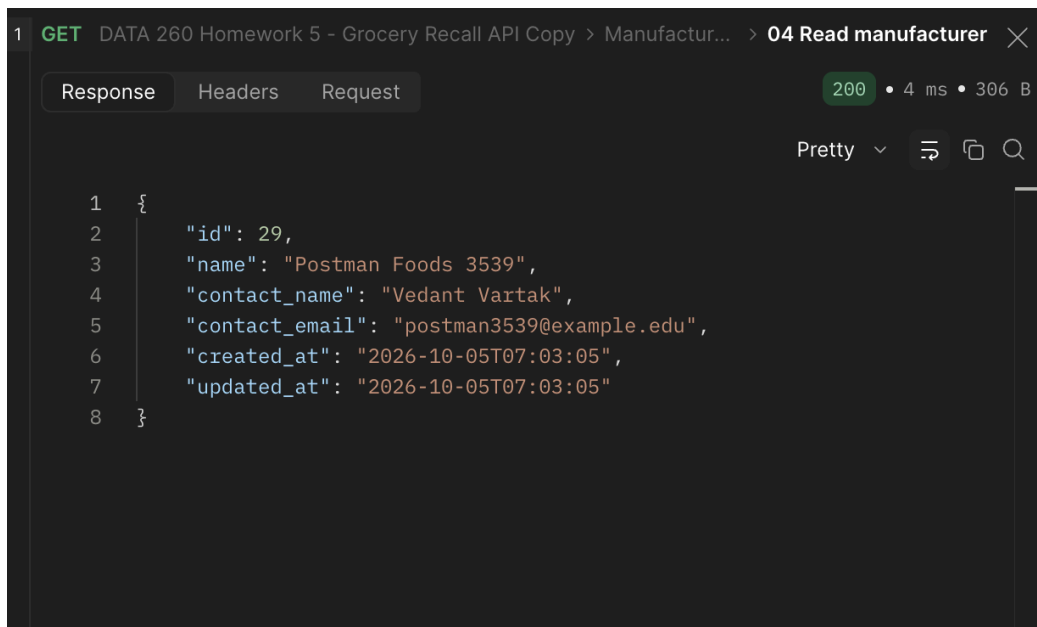
Cropped response pane: returned recall 6008 and its manufacturer relation; original capture is retained.

5 Actual Postman responses list and direct record read

The list request demonstrates the fixed eager-load implementation and page metadata. The next capture is a completed direct-read response for the manufacturer used by the recall workflow; Section 3 independently records the matching direct recall read.



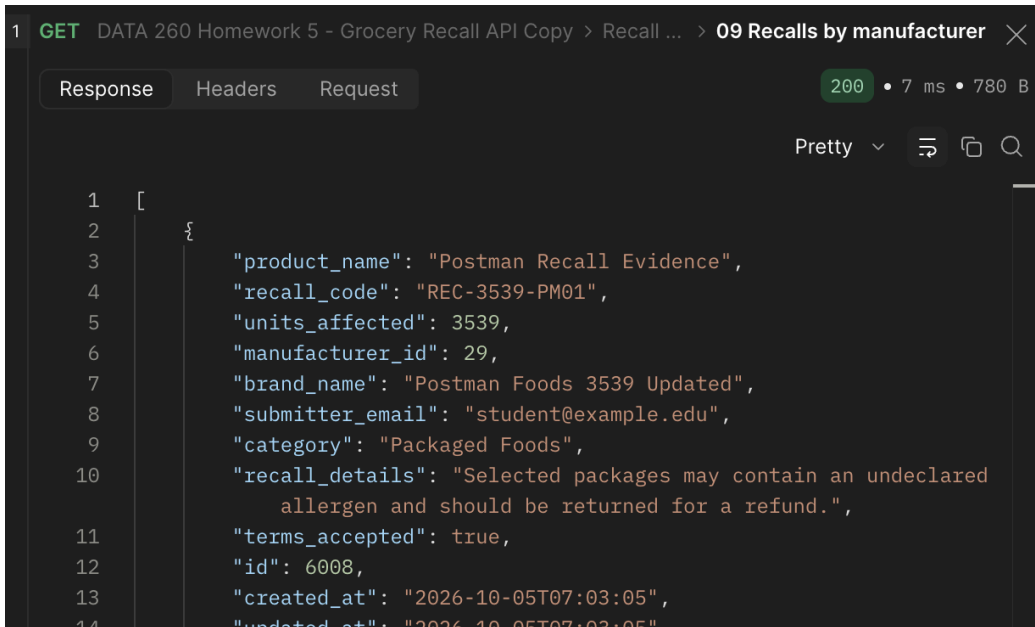
Enlarged list response pane: implementation metadata, total, page size, and records; original is retained.



Enlarged direct manufacturer-read response pane; original is retained.

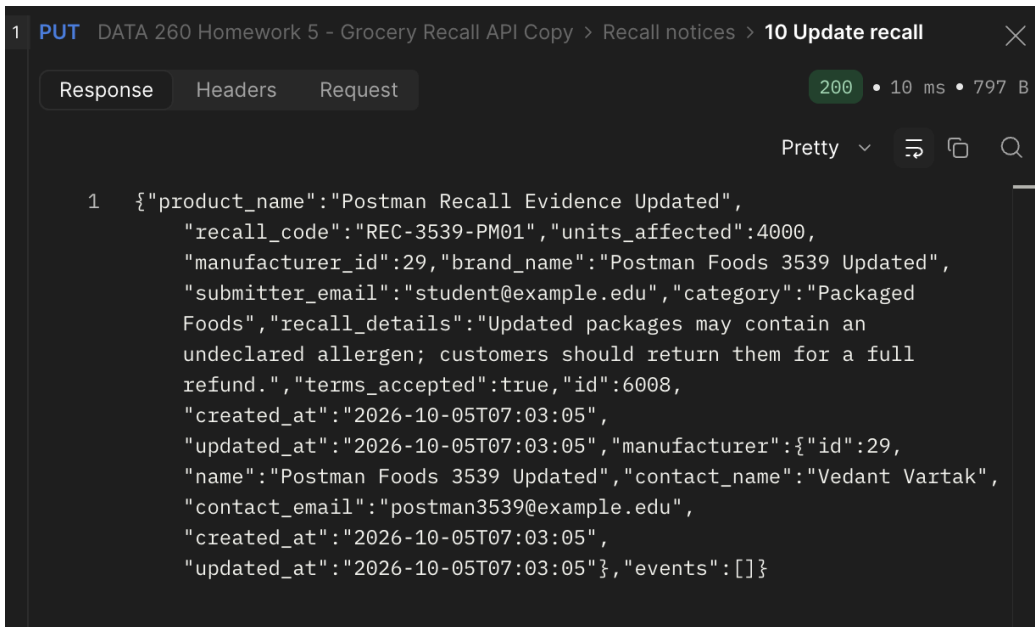
6 Actual Postman responses relationship update and delete

The relationship query and update are captured directly in Postman. The Runner capture in Section 3 records the subsequent 204 delete response for this same recall, since a successful DELETE has no response body to display.



```
1 GET DATA 260 Homework 5 - Grocery Recall API Copy > Recall ... > 09 Recalls by manufacturer X
Response Headers Request 200 • 7 ms • 780 B
Pretty ↕ ↗ 🔍
1 [
2   {
3     "product_name": "Postman Recall Evidence",
4     "recall_code": "REC-3539-PM01",
5     "units_affected": 3539,
6     "manufacturer_id": 29,
7     "brand_name": "Postman Foods 3539 Updated",
8     "submitter_email": "student@example.edu",
9     "category": "Packaged Foods",
10    "recall_details": "Selected packages may contain an undeclared
11                      allergen and should be returned for a refund.",
12    "terms_accepted": true,
13    "id": 6008,
14    "created_at": "2026-10-05T07:03:05",
15    "updated_at": "2026-10-05T07:03:05"
```

Enlarged relationship response pane, including recall 6008; original is retained.



```
1 PUT DATA 260 Homework 5 - Grocery Recall API Copy > Recall notices > 10 Update recall X
Response Headers Request 200 • 10 ms • 797 B
Pretty ↕ ↗ 🔍
1 {"product_name":"Postman Recall Evidence Updated",
  "recall_code":"REC-3539-PM01","units_affected":4000,
  "manufacturer_id":29,"brand_name":"Postman Foods 3539 Updated",
  "submitter_email":"student@example.edu","category":"Packaged
  Foods","recall_details":"Updated packages may contain an
  undeclared allergen; customers should return them for a full
  refund.","terms_accepted":true,"id":6008,
  "created_at":"2026-10-05T07:03:05",
  "updated_at":"2026-10-05T07:03:05","manufacturer":{"id":29,
  "name":"Postman Foods 3539 Updated","contact_name":"Vedant Vartak",
  "contact_email":"postman3539@example.edu",
  "created_at":"2026-10-05T07:03:05",
  "updated_at":"2026-10-05T07:03:05"},"events":[]}
```

Enlarged updated-recall response pane; original is retained. The Runner records subsequent DELETE 204.

6A Actual Postman response manufacturer deletion

This final cleanup action is shown in the real Postman Runner after its dependent recall was deleted. HTTP 204 deliberately has no response body; the completed request and status are both visible.

The screenshot displays the Postman Runner interface. On the left, a sidebar shows a collection of API tests under 'DATA 260 Homework 5 - Grocery Recall API'. The main panel shows the 'Run results' for a specific run. A table summarizes the test results:

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	DATA 260 HW5 Local 3539	1	536ms	2	0	16 ms

Below the table, a list of test results is shown. The 'DELETE 12 Delete manufacturer' test is highlighted, showing a 204 status code. The detailed view of this test is shown on the right, indicating that the response does not have a body.

DELETE 12 Delete manufacturer
http://127.0.0.1:8839/api/v1/manufacturers/29
204

This response doesn't have a body.

DELETE /api/v1/manufacturers/29 returned 204 No Content in Postman, completing the temporary demonstration data cleanup.

7 Enlarged live MySQL evidence

The database capture is given a full page so the row counts, schema, and foreign key are readable.

```
Last login: Sun Oct 4 00:34:07 on ttys018
cd /Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539 && clear && .venv/bin/python scripts/capture_hw05_database_evidence.py
vedantvartak@Vedants-MacBook-Air ~ % cd /Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539 && clear && .venv/bin/python scripts/capture_hw05_database_evidence.py

VEDANT VARTAK - DATA 260 HOMEWORK 5 - MYSQL EVIDENCE
timestamp.Utc: 2026-10-04T07:34:11.556960+00:00
database: s3539.sql
tables: manufacturers, recall_events, recall_notices, sessions, users

row_counts:
  manufacturers: 12
  recall_notices: 500
  recall_events: 200
  users: 1
  sessions: 1

manufacturers_columns:
  id | INTEGER | nullable=False
  name | VARCHAR(160) | nullable=False
  contact_name | VARCHAR(160) | nullable=False
  contact_email | VARCHAR(255) | nullable=False
  created_at | DATETIME | nullable=False
  updated_at | DATETIME | nullable=False

recall_notices_columns:
  id | INTEGER | nullable=False
  product_name | VARCHAR(160) | nullable=False
  recall_code | VARCHAR(32) | nullable=False
  units_affected | INTEGER | nullable=False
  manufacturer_id | INTEGER | nullable=False
  brand_name | VARCHAR(160) | nullable=False
  submitter_email | VARCHAR(255) | nullable=False
  category | VARCHAR(60) | nullable=False
  recall_details | TEXT | nullable=False
  terms_accepted | TINYINT | nullable=False
  created_at | DATETIME | nullable=False
  updated_at | DATETIME | nullable=False

recall_notices_foreign_keys:
  fk_recall_notices_manufacturer: ['manufacturer_id'] -> manufacturers['id'] onDelete=RESTRICT
Saved: /Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539/reports/hw05/raw/database_evidence.txt
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

12 manufacturers, 500 recalls, 200 events, required columns, and RESTRICT foreign key.

7A Database evidence readable detail

This close crop retains the row counts and schema columns while removing unused terminal space. The preceding page remains the complete, unmodified capture.

```
VEDANT VARTAK - DATA 260 HOMEWORK 5 - MYSQL EVIDENCE
timestamp_utc: 2026-10-04T07:34:11.556960+00:00
database: s3539_rel
tables: manufacturers, recall_events, recall_notices, sessions, users

row_counts:
  manufacturers: 12
  recall_notices: 500
  recall_events: 200
  users: 1
  sessions: 1

manufacturers_columns:
  id | INTEGER | nullable=False
  name | VARCHAR(160) | nullable=False
  contact_name | VARCHAR(160) | nullable=False
  contact_email | VARCHAR(255) | nullable=False
  created_at | DATETIME | nullable=False
  updated_at | DATETIME | nullable=False

recall_notices_columns:
  id | INTEGER | nullable=False
  product_name | VARCHAR(160) | nullable=False
  recall_code | VARCHAR(32) | nullable=False
  units_affected | INTEGER | nullable=False
  manufacturer_id | INTEGER | nullable=False
  brand_name | VARCHAR(160) | nullable=False
  submitter_email | VARCHAR(255) | nullable=False
  category | VARCHAR(80) | nullable=False
  recall_details | TEXT | nullable=False
  terms_accepted | TINYINT | nullable=False
  created_at | DATETIME | nullable=False
  updated_at | DATETIME | nullable=False

recall_notices_foreign_keys:
  fk_recall_notices_manufacturer: ['manufacturer_id'] -> manufacturers['id'] ondelete=RESTRICT
Saved: /Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539/reports/hw05/raw/database_evidence.tx
vedantvartak@Vedants-MacBook-Air: data260-3539 %
```

Close view of the recorded MySQL row counts and required schema columns.

8 Database and API implementation evidence

The relevant validation and transaction code is immediately followed by measured output.

DATABASE + API VALIDATION CODE AND OUTPUT

IMPLEMENTATION

RecallCreate validates positive units, email, exact category, accepted terms, unique recall_code, and manufacturer_id. The API catches IntegrityError and returns HTTP 409; the FK uses ON DELETE RESTRICT.

CODE

```
class RecallCreate(RecallInput): pass

if db.get(Manufacturer, payload.manufacturer_id) is None:
    raise HTTPException(404, 'Manufacturer not found')
try: db.commit()
except IntegrityError:
    db.rollback(); raise HTTPException(409, 'Recall code must be unique')
```

MEASURED OUTPUT

```
manufacturers=12; recall_notices=500; recall_events=200
FK: manufacturer_id -> manufacturers.id ON DELETE RESTRICT
14/14 authenticated live API checks passed
```

Validation, 409 handling, relationship integrity, and live row counts.

9 Authentication format validation and numeric default

The requested security and schema details are explicit rather than inferred from PASS claims.

PASSWORD, UNIQUE FORMAT, AND NUMERIC DEFAULT

PASSWORD HASHING

PBKDF2-HMAC-SHA256; 240,000 iterations; random 16-byte salt; constant-time hmac.compare_digest verification. Only password_hash is stored.

UNIQUE FIELD FORMAT

recall_code: Field(pattern=r'^REC-[A-Z0-9]{2,23}\$'); normalized with strip().upper(). Database column is unique=True and indexed. contact_email and submitter_email use EmailStr.

NUMERIC DEFAULT

units_affected: int = Field(default=0, ge=0, le=100_000_000)
SQLAlchemy: mapped_column(Integer, nullable=False, default=0)

Salted PBKDF2 hashing, unique-field formats, and the units_affected default are documented from source.

9A Actual password hashing function

Exact source excerpt: `code/web_application/security.py`. The measured result is immediately below.

```
PBKDF2_ITERATIONS = 240_000
```

```
def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.pbkdf2_hmac(
        "sha256",
        password.encode("utf-8"),
        salt,
        PBKDF2_ITERATIONS,
    )
    return "$".join(
        (
            "pbkdf2_sha256",
            str(PBKDF2_ITERATIONS),
            base64.urlsafe_b64encode(salt).decode("ascii"),
            base64.urlsafe_b64encode(digest).decode("ascii"),
        )
    )
```

Verified output

```
Different salts produce different hashes: True
Correct password verifies: True
Incorrect password rejected: True
```

10 Redux store and slice setup

The store registration, async thunk, and slice state are shown as implementation evidence.

REDUX STORE AND SLICE IMPLEMENTATION

STORE

```
export const store = configureStore({
  reducer: { recalls: recallsReducer },
});

<Provider store={store}><App /></Provider>
```

ASYNC THUNK

```
export const fetchRecalls = createAsyncThunk(
  'recalls/fetch',
  async ({ query = '', page = 1 } = {}) => {
    const response = await apiClient.get('/recalls',
      { params: { q: query || undefined, page, page_size: 50 } });
    return response.data;
  }
);
```

SLICE STATE

```
initialState: { items: [], page: 1, pageSize: 50, total: 0, status: 'idle' }
fulfilled: state.items = action.payload.records; state.total = action.payload.total
```

configureStore registers the recalls reducer; the thunk and fulfilled reducer preserve pagination metadata.

11 Redux Home code and UI in one screenshot

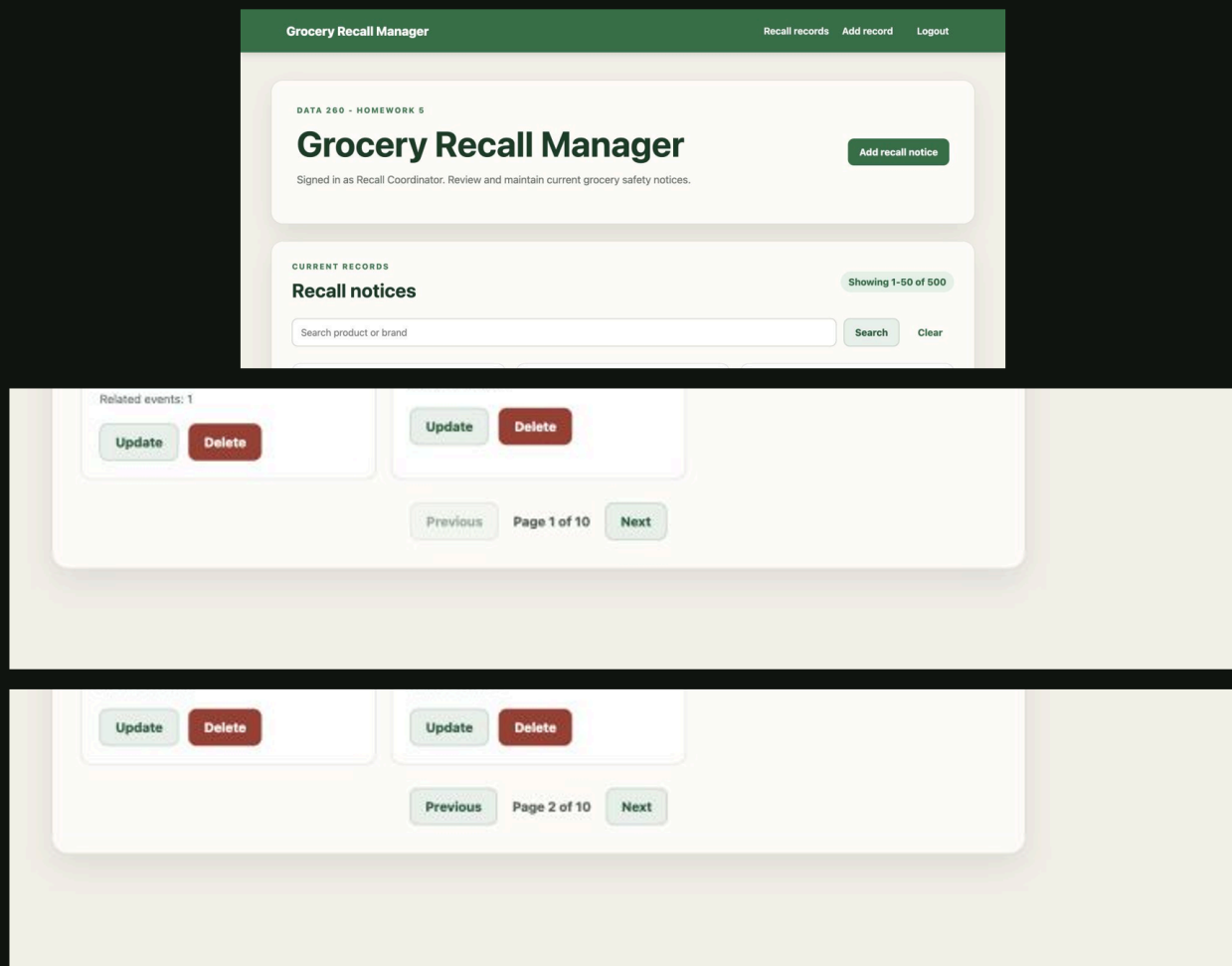
The composite contains the actual thunk/slice fields and visible UI output, including page 1 and page 2.

REDUX HOME: PAGINATED ACCESS TO ALL 500 RECORDS

RELEVANT REDUX / REACT CODE

```
export const fetchRecalls = createAsyncThunk('recalls/fetch',
  async ({ query = '', page = 1 } = {}, thunkApi) => {
    const params = { page, page_size: 50 };
    const response = await apiClient.get('/recalls', { params });
    return response.data;
  });

const { items, page, pageSize, total } = useSelector(s => s.recalls);
<span>Showing {firstShown}-{lastShown} of {total}</span>
<span>Page {page} of {totalPages}</span>
```



Showing 1-50 of 500; page 1/10 and page 2/10 prove access beyond the first 50 records.

12 Redux Create form code and saved output

The create form, thunk, and persisted success result appear in one report frame.

REDUX CREATE: THUNK + SAVED UI OUTPUT

RELEVANT REDUX / REACT CODE

```
await dispatch(createRecall(payload)).unwrap();
export const createRecall = createAsyncThunk('recalls/create', async (payload) => {
  const response = await apiClient.post('/recalls', payload);
  return response.data;
});
```

The image displays two screenshots of a web application interface. The top screenshot shows a form titled 'Add a recall notice' with the subtitle 'Enter the affected product and recall information.' The form contains several input fields: 'Product name' (filled with 'HWS Evidence Granola Bars'), 'Recall code (unique)' (filled with 'REC-3539-90001'), 'Units affected' (filled with '03539'), 'Manufacturer' (a dropdown menu showing 'Grocery Safety Manufacturer 01'), 'Brand or manufacturer' (filled with 'Campus Pantry Evidence'), 'Submitter email' (filled with 'student@example.edu'), and 'Product category' (a dropdown menu showing 'Packaged Foods'). There is also a 'Recall details' text area containing the text 'Selected packages may contain an undeclared peanut allergen and should be returned for a full refund.' At the bottom of the form, there is a checkbox labeled 'I agree to the terms and conditions.' which is checked. The bottom screenshot shows the result of the form submission. It features a green success message at the top: 'Added recall REC-3539-90001 successfully.' Below this, there is a section titled 'CURRENT RECORDS' with the heading 'Recall notices' and a '1 shown' indicator. A search bar contains the text 'HWS Evidence' with 'Search' and 'Clear' buttons. The search results show a single entry with the following details: '#0006', 'HWS Evidence Granola Bars', 'REC-3539-90001', 'Campus Pantry Evidence', '3,539 units affected', and the same recall details text. At the bottom of the entry, it says 'Related events: 0' and has 'Update' and 'Delete' buttons. A 'Packaged Foods' tag is visible on the right side of the entry.

The completed form is visible before createRecall returns the saved record and success notice.

13 Redux Update form code and revised output

The update form, thunk/reducer, and revised record output appear together.

REDUX UPDATE: THUNK + UPDATED UI OUTPUT

RELEVANT REDUX / REACT CODE

```
await dispatch(updateRecall({ id: Number(id), payload })).unwrap();
.addCase(updateRecall.fulfilled, (state, action) => {
  const index = state.items.findIndex(item => item.id === action.payload.id);
  if (index >= 0) state.items[index] = action.payload;
});
```

The first screenshot shows the 'Update recall #6006' form. It contains fields for Product name (HWS Evidence Granola Bars Updated), Recall code (unique) (REC-3539-90001), Units affected (4540), Manufacturer (Grocery Safety Manufacturer 01), Brand or manufacturer (Campus Pantry Evidence Updated), and Submitter email (student@example.edu). The Product category is set to Packaged Foods. The Recall details section contains the text: 'Updated packages may contain an undeclared peanut allergen. Customers should return them for a full refund.' A checkbox for 'I agree to the terms and conditions.' is checked.

The second screenshot shows the 'Recall notices' section after the update. A green message bar at the top says 'Updated recall REC-3539-90001 successfully.' Below the 'Recall notices' header, there is a search bar and a 'Search' button. The list shows one record: #6006, HWS Evidence Granola Bars Updated, REC-3539-90001, Campus Pantry Evidence Updated, 4540 units affected, and the same recall details. The record is categorized as 'Packaged Foods'. At the bottom of the record, there are 'Update' and 'Delete' buttons.

The edited form is visible before Redux replaces the item with the API response.

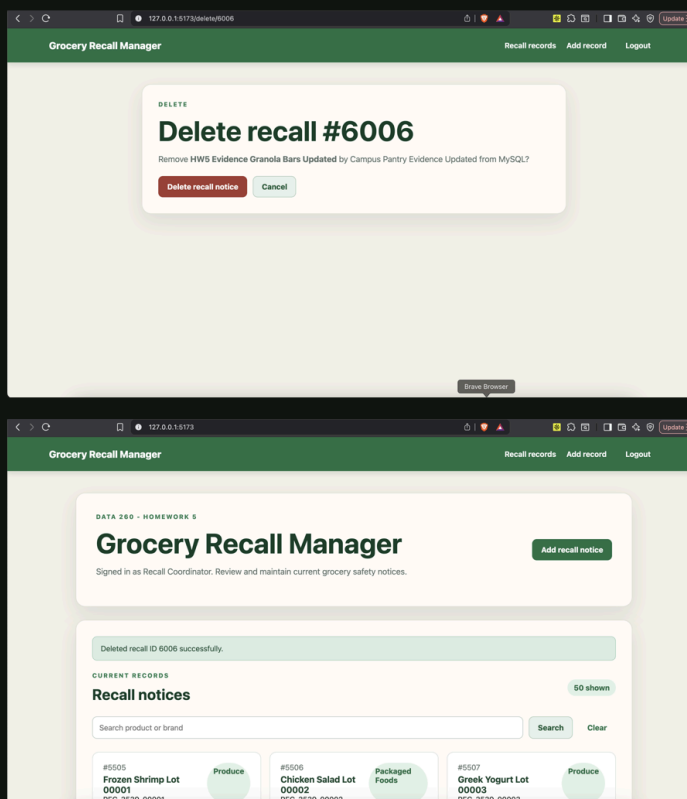
14 Redux Delete code confirmation and output

The dedicated confirmation route and post-delete state appear with the reducer code in one composite screenshot.

REDUX DELETE: CONFIRMATION ROUTE + UI OUTPUT

RELEVANT REDUX / REACT CODE

```
<Link to={`/delete/${record.id}`}>Delete</Link>
await dispatch(deleteRecall(Number(id)).unwrap());
.addCase(deleteRecall.fulfilled, (state, action) => {
  state.items = state.items.filter(item => item.id !== action.payload);
  state.notice = `Deleted recall ID ${action.payload} successfully.`;
});
```



Explicit confirmation precedes deletion; the success notice follows API completion.

Part 2A MealDB MCP implementation

This source evidence shows the exact four-tool surface, stdio transport, stderr logging, HTTP timeout, and handled failures.

MEALDB MCP SERVER IMPLEMENTATION

STDIO + STDERR

```
logging.basicConfig(stream=sys.stderr, level=logging.INFO)
mcp = FastMCP('meals')
if __name__ == '__main__': mcp.run(transport='stdio')
```

TIMEOUT + ERROR HANDLING

```
try:
    response = httpx.get(url, params=params, timeout=10.0)
    response.raise_for_status()
    payload = response.json()
except (httpx.HTTPError, ValueError) as exc:
    LOGGER.error('TheMealDB request failed: %s', exc)
    raise RuntimeError(f'TheMealDB request failed: {exc}') from exc
```

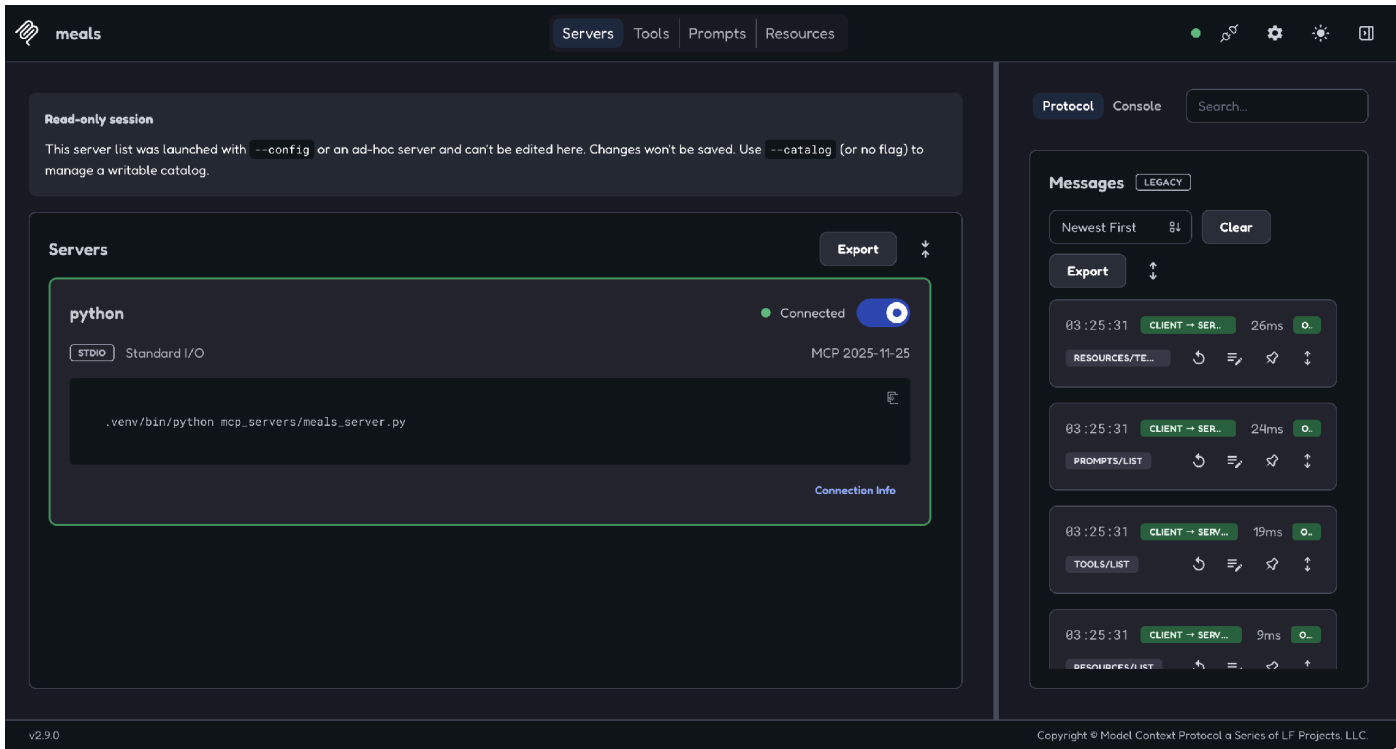
EXACT TOOL SURFACE

```
@mcp.tool search_meals_by_name
@mcp.tool meals_by_ingredient
@mcp.tool meal_details
@mcp.tool random_meal
```

MealDB server implementation excerpts and failure boundary.

16 MealDB MCP server connection

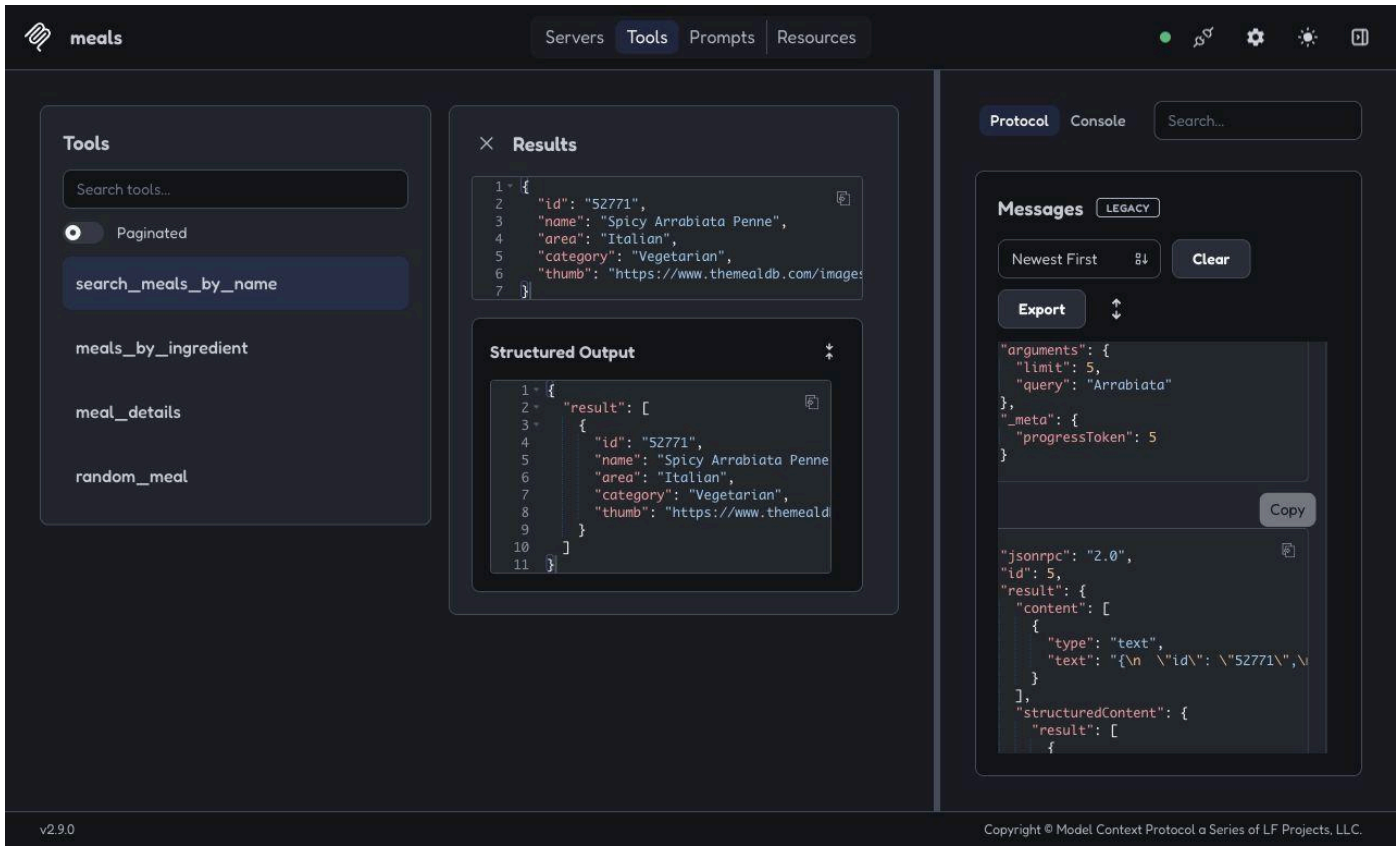
The Inspector connected over stdio using the project virtual environment and exposed exactly four required tools.



Connected MealDB MCP server in Inspector; tools, prompts, and resources inventories completed successfully.

17 MealDB Inspector search_meals_by_name

The actual Inspector view visibly includes the submitted arguments and returned output.



Actual MCP Inspector call for search_meals_by_name, with the Protocol pane expanded to show its exact arguments and response.

17A MealDB result search_meals_by_name

The exact companion panel is placed on its own page so the result is readable at normal viewing size.

MEALDB MCP TOOL 1/4: search_meals_by_name

INPUT JSON

```
{
  "query": "Arrabiata",
  "limit": 5
}
```

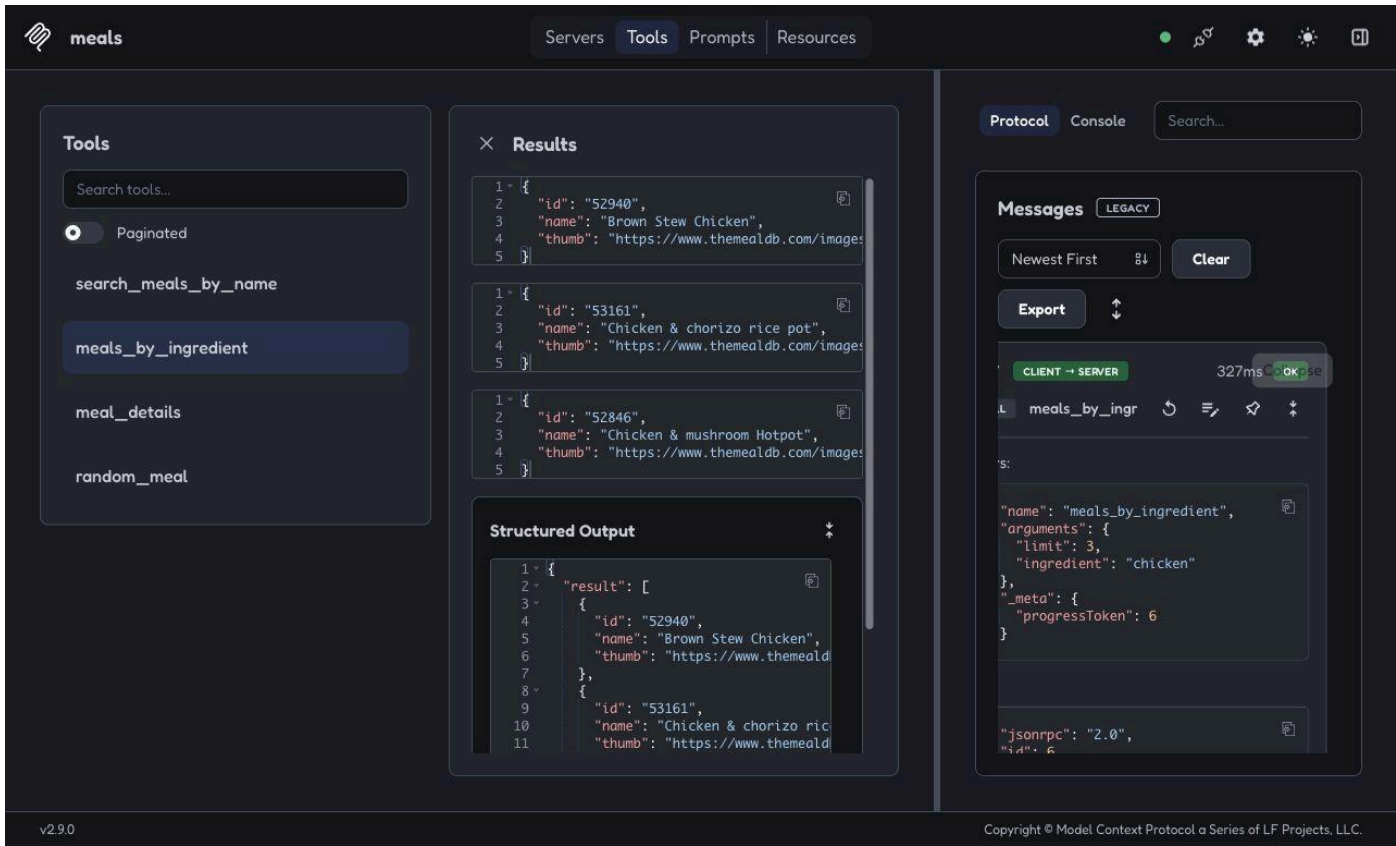
STRUCTURED OUTPUT

```
{
  "result": [
    {
      "id": "52771",
      "name": "Spicy Arrabiata Penne",
      "area": "Italian",
      "category": "Vegetarian",
      "thumb": "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg"
    }
  ]
}
```

Exact input JSON and readable output for search_meals_by_name; complete raw output is retained.

18 MealDB Inspector meals_by_ingredient

The actual Inspector view visibly includes the submitted arguments and returned output.



Actual MCP Inspector call for meals_by_ingredient, with the Protocol pane expanded to show its exact arguments and response.

18A MealDB result meals_by_ingredient

The exact companion panel is placed on its own page so the result is readable at normal viewing size.

MEALDB MCP TOOL 2/4: meals_by_ingredient

INPUT JSON

```
{
  "ingredient": "chicken",
  "limit": 3
}
```

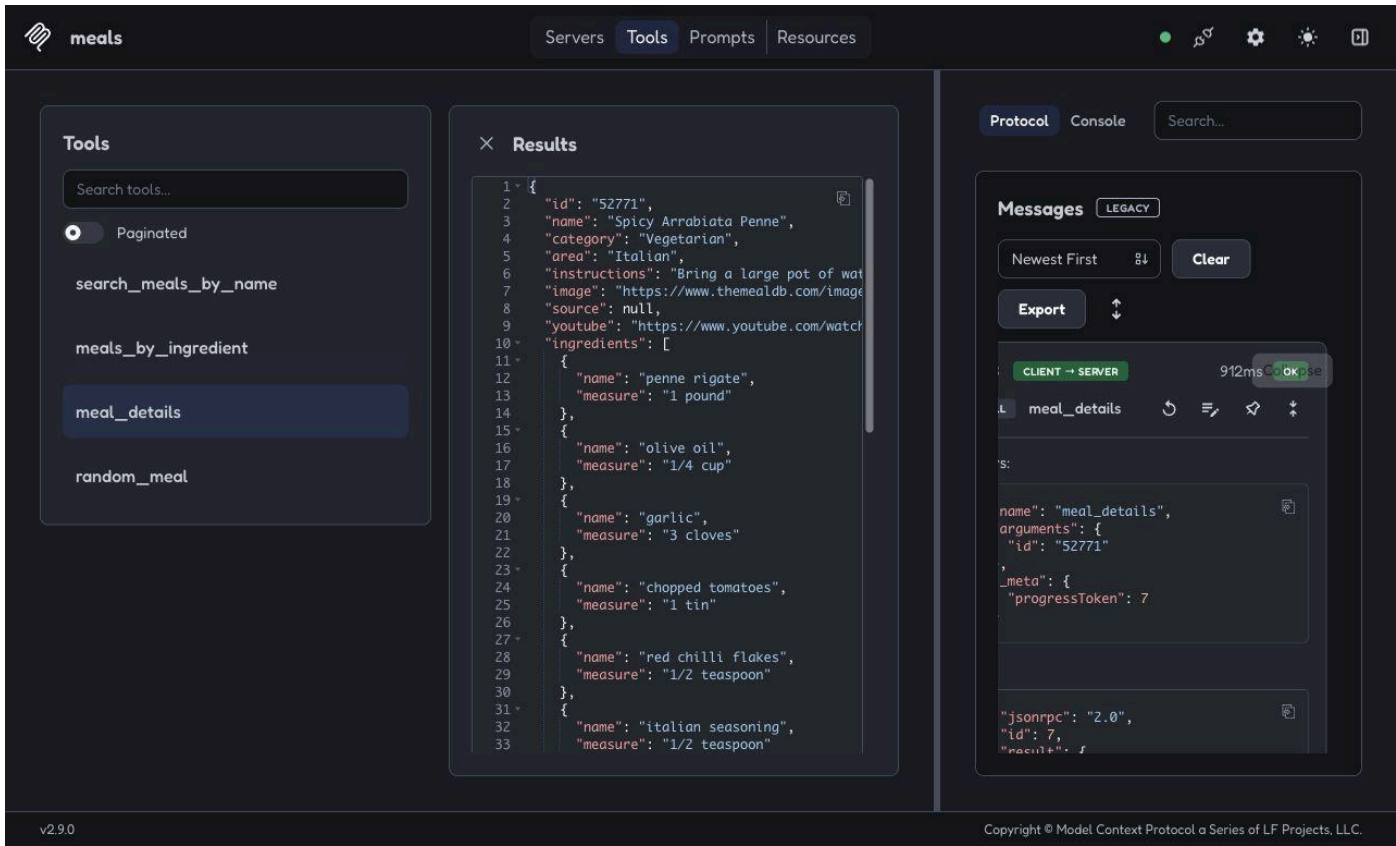
STRUCTURED OUTPUT

```
{
  "result": [
    {
      "id": "52940",
      "name": "Brown Stew Chicken",
      "thumb": "https://www.themealdb.com/images/media/meals/sypxpx1515365095.jpg"
    },
    {
      "id": "53161",
      "name": "Chicken & chorizo rice pot",
      "thumb": "https://www.themealdb.com/images/media/meals/fk80jp1763280767.jpg"
    },
    {
      "id": "52846",
      "name": "Chicken & mushroom Hotpot",
      "thumb": "https://www.themealdb.com/images/media/meals/uuuspp1511297945.jpg"
    }
  ]
}
```

Exact input JSON and readable output for meals_by_ingredient; complete raw output is retained.

19 MealDB Inspector meal_details

The actual Inspector view visibly includes the submitted arguments and returned output.



Actual MCP Inspector call for meal_details, with the Protocol pane expanded to show its exact arguments and response.

19A MealDB result meal_details

The exact companion panel is placed on its own page so the result is readable at normal viewing size.

MEALDB MCP TOOL 3/4: meal_details

INPUT JSON

```
{
  "id": "52771"
}
```

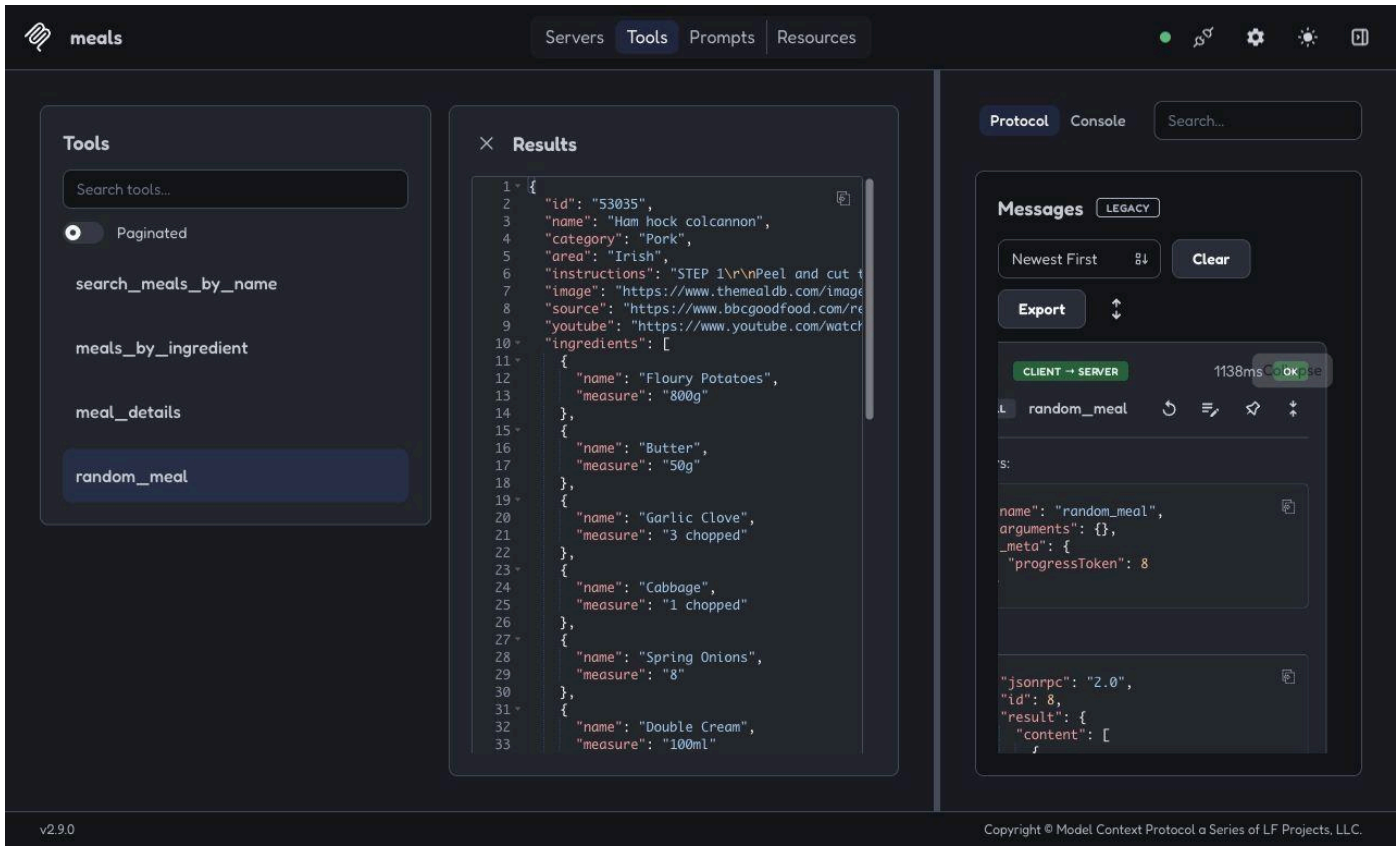
STRUCTURED OUTPUT

```
{
  "id": "52771",
  "name": "Spicy Arrabiata Penne",
  "category": "Vegetarian",
  "area": "Italian",
  "instructions": "Bring a large pot of water to a boil. Add kosher salt to the boiling water, then add the pasta. Cook according to the package instructions, about 9 minutes.\r\n\r\nIn a large skillet over medium-high heat, add the olive oil and heat until the oil starts to shimmer. Add the garlic and cook, stirring, until fragrant, 1 to 2 minutes. Add the chopped tomatoes, red chile flakes, Italian seasoning and salt and pepper to taste. Bring to a boil and cook for 5 minutes. Remove from the heat and add the chopped basil.\r\n\r\nDrain the pasta and add it to the sauce. Garnish with Parmigiano-Reggiano flakes and more basil and serve warm.",
  "image": "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg",
  "source": null,
  "youtube": "https://www.youtube.com/watch?v=1IszT_gul08",
  "ingredients": [
    {"name": "penne rigate", "measure": "1 pound"},
    {"name": "olive oil", "measure": "1/4 cup"},
    {"name": "garlic", "measure": "3 cloves"},
    {"name": "chopped tomatoes", "measure": "1 tin"},
    {"name": "red chilli flakes", "measure": "1/2 teaspoon"},
    {"name": "italian seasoning", "measure": "1/2 teaspoon"},
    {"name": "basil", "measure": "6 leaves"},
    {"name": "Parmigiano-Reggiano", "measure": "sprinkling"}
  ]
}
```

Exact input JSON and readable output for meal_details; complete raw output is retained.

20 MealDB Inspector random_meal

The actual Inspector view visibly includes the submitted arguments and returned output.



Actual MCP Inspector call for random_meal, with the Protocol pane expanded to show its exact arguments and response.

20A MealDB result random_meal

The exact companion panel is placed on its own page so the result is readable at normal viewing size.

MEALDB MCP TOOL 4/4: random_meal

INPUT JSON

```
{}
```

STRUCTURED OUTPUT

```
{
  "id": "53035",
  "name": "Ham hock colcannon",
  "category": "Pork",
  "area": "Irish",
  "instructions": "STEP 1\nPeel and cut the potatoes into even, medium-sized chunks. Put in a large pan filled with cold salted water, bring to the boil and cook for 10-15 mins until a knife can be inserted into the potatoes easily.\n\nSTEP 2\nMeanwhile, melt the butter in a large sauté pan over a medium heat. Add the garlic, cabbage, spring onions and some seasoning. Stir occasionally until the cabbage is wilted but still retains a little bite, then set aside.\n\nSTEP 3\nDrain the potatoes, leave to steam-dry for a couple of mins, then mash with the cream, mustard and seasoning in the same saucepan. Stir in the cabbage and ham hock. Keep warm over a low heat.\n\nSTEP 4\nReheat the pan you used to cook the cabbage (no need to wash first), add a splash of oil, crack in the eggs and fry to your liking. To serve, divide the colcannon between bowls and top each with a fried egg.",
  "image": "https://www.themealdb.com/images/media/meals/n41ny81608588066.jpg",
  "source": "https://www.bbcgoodfood.com/recipes/ham-hock-colcannon",
  "youtube": "https://www.youtube.com/watch?v=aQJAU2iJ730",
  "ingredients": [
    {"name": "Floury Potatoes", "measure": "800g"},
    {"name": "Butter", "measure": "50g"},
    {"name": "Garlic Clove", "measure": "3 chopped"},
    {"name": "Cabbage", "measure": "1 chopped"},
    {"name": "Spring Onions", "measure": "8"},
    {"name": "Double Cream", "measure": "100ml"},
    {"name": "Mustard", "measure": "2 tbs"},
    {"name": "Ham", "measure": "180g"},
    {"name": "Eggs", "measure": "4"}
  ]
}
```

Exact input JSON and readable output for random_meal; complete raw output is retained.

Part 2B Domain MCP implementation

The server exposes exactly three stdio tools and routes all calls through the validated shared boundary.

DOMAIN MCP SERVER IMPLEMENTATION

STDIO + STDERR

```
logging.basicConfig(stream=sys.stderr, level=logging.INFO)
mcp = FastMCP('s3539_rel-recall-tools')
if __name__ == '__main__': mcp.run(transport='stdio')
```

EXACT THREE TOOLS

```
@mcp.tool(name='search')
def search(query: str, limit: int = 5): ...
@mcp.tool(name='detail')
def detail(recall_id: int): ...
@mcp.tool(name='aggregate')
def aggregate(group_by: str, min_units: int = 0): ...
```

BOUNDARY ERROR ENVELOPE

```
call_domain_tool validates typed inputs and catches ValidationError, returning the same JSON-serializable
{ok, data, error} shape for success and failure.
```

stderr logging, stdio transport, exact tool signatures, and stable error envelope.

22 Actual Domain Inspector search valid

A full-page close crop enlarges the actual result and Protocol input panes. Original screenshots are retained; the following contract page supplies untruncated JSON.

The screenshot displays the Domain Inspector interface with a valid search call. The interface is divided into two main sections: the left pane for results and the right pane for protocol and messages.

Left Pane:

- Results:** Displays a JSON response with the following structure:

```
1 {
2   "ok": true,
3   "data": {
4     "matches": [
5       {
6         "id": 5505,
7         "recall_code": "REC-3539-00001",
8         "product_name": "Frozen Shrimp Lot",
9         "brand_name": "Grocery Safety Manuf",
10        "category": "Produce",
11        "units_affected": 6535
12      }
13    ],
14    "count": 1
15  },
16  "error": null
17 }
```
- Structured Output:** Displays the same JSON response as the Results section.

Right Pane:

- Protocol:** Shows the request and response details.
- Console:** Shows the messages section with a search call.
- Messages:** Displays the request and response details for the search call. The request is a POST to the search endpoint with the following parameters:

```
1 {
2   "name": "search",
3   "arguments": {
4     "limit": 1,
5     "query": "shrimp"
6   },
7   "_meta": {
8     "progressToken": 6
9   }
10 }
```

The response is a JSON object with the following structure:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 6,
4   "result": {
5     "content": [
6       {
7         "id": 5505,
8         "recall_code": "REC-3539-00001",
9         "product_name": "Frozen Shrimp Lot",
10        "brand_name": "Grocery Safety Manuf",
11        "category": "Produce",
12        "units_affected": 6535
13      }
14     ],
15     "count": 1
16   }
17 }
```

Actual valid search call. This crop changes only framing, not submitted inputs or returned output.

22 Actual Domain Inspector search invalid

A full-page close crop enlarges the actual result and Protocol input panes. Original screenshots are retained; the following contract page supplies untruncated JSON.

The screenshot displays the Domain Inspector interface with two main panes. The left pane, titled 'Results', shows a JSON object with an error message: "Invalid search input: query: Str". Below this, the 'Structured Output' section displays the same JSON object. The right pane, titled 'Messages', shows a message from the client to the server at 05:51:51, 33ms, with a 'collapse' button. The message details include parameters and a response.

Results

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "Invalid search input: query: Str"
5 }
```

Structured Output

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "Invalid search input: que"
5 }
```

Messages LEGACY

Newest First 32 Clear

Export

05:51:51 CLIENT -> SER... 33ms collapse

TOOLS/CALL sei ↺ ⋮ ⌂ ⚙

Parameters:

```
1 {
2   "name": "search",
3   "arguments": {
4     "limit": 5,
5     "query": "s"
6   },
7   "_meta": {
8     "progressToken": 7
9   }
10 }
```

Response:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 7,
4   "result": {
5     "content": [
6     ]
7   }
8 }
```

Actual invalid search call. This crop changes only framing, not submitted inputs or returned output.

23 Domain MCP contract search

Expected JSON schema, valid input/output, rejected JSON, complete error, and rejection reason are shown together.

DOMAIN MCP CONTRACT 1/3: search

EXPECTED JSON INPUT SCHEMA

```
{
  "type": "object",
  "additionalProperties": false,
  "required": ["query"],
  "properties": {
    "query": {
      "type": "string",
      "minLength": 2,
      "maxLength": 80
    },
    "limit": {
      "type": "integer",
      "minimum": 1,
      "maximum": 25,
      "default": 5
    }
  }
}
```

VALID INPUT

```
{
  "query": "shrimp",
  "limit": 1
}
```

VALID OUTPUT

```
{
  "ok": true,
  "data": {
    "matches": [
      {
        "id": 5505,
        "recall_code": "REC-3539-00001",
        "product_name": "Frozen Shrimp Lot 00001",
        "brand_name": "Grocery Safety Manufacturer 10",
        "category": "Produce",
        "units_affected": 6535
      }
    ],
    "count": 1
  },
  "error": null
}
```

REJECTED JSON OBJECT

```
{
  "query": "s",
  "limit": 5
}
```

COMPLETE RETURNED ERROR

```
{
  "ok": false,
  "data": null,
  "error": "Invalid search input: query: String should have at least 2 characters"
}
```

WHY REJECTED

One character is below the declared minimum and creates an excessively broad lookup.

The search contract returns the stable {ok, data, error} envelope for both valid and invalid calls.

24 Actual Domain Inspector detail valid

A full-page close crop enlarges the actual result and Protocol input panes. Original screenshots are retained; the following contract page supplies untruncated JSON.

Results

```
1 {
2   "ok": true,
3   "data": {
4     "id": 5505,
5     "recall_code": "REC-3539-00001",
6     "product_name": "Frozen Shrimp Lot 0000",
7     "brand_name": "Grocery Safety Manufactu",
8     "category": "Produce",
9     "units_affected": 6535,
10    "recall_details": "Grocery Safety Manu",
11    "manufacturer": {
12      "id": 24,
13      "name": "Grocery Safety Manufacturer",
14      "contact_name": "Safety Contact 10"
15    }
16  },
17  "error": null
18 }
```

Structured Output

```
1 {
2   "ok": true,
3   "data": {
4     "id": 5505,
5     "recall_code": "REC-3539-00001",
6     "product_name": "Frozen Shrimp L",
7     "brand_name": "Grocery Safety Ma",
8     "category": "Produce",
9     "units_affected": 6535,
10    "recall_details": "Grocery Safet",
11    "manufacturer": {
12      "id": 24,
13      "name": "Grocery Safety Manufa",
14      "contact_name": "Safety Contac
```

Protocol Console Search...

Messages LEGACY

Newest First Clear

Export

05:52:10 CLIENT -> SER... 36ms close

TOOLS/CALL de ↺ ↻ ↶ ↷

Parameters:

```
1 {
2   "name": "detail",
3   "arguments": {
4     "recall_id": 5505
5   },
6   "_meta": {
7     "progressToken": 8
8   }
9 }
```

Response:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 8,
4   "result": {
5     "content": [
6       {
7         "type": "text".
```

Actual valid detail call. This crop changes only framing, not submitted inputs or returned output.

24 Actual Domain Inspector detail invalid

A full-page close crop enlarges the actual result and Protocol input panes. Original screenshots are retained; the following contract page supplies untruncated JSON.

The screenshot displays the 'Actual Domain Inspector' interface, which is divided into two main panels: 'Results' on the left and 'Messages' on the right.

Results Panel:

- Results:** A code editor showing the following JSON object:

```
1 {  
2   "ok": false,  
3   "data": null,  
4   "error": "Invalid detail input: recall_id"  
5 }
```
- Structured Output:** A code editor showing the same JSON object as above.

Messages Panel:

- Protocol:** A tab selected at the top.
- Console:** A search bar is present.
- Messages:** A section with a 'LEGACY' button and a 'Clear' button.
- Export:** A button with a download icon.
- Message Details:** A message is shown with a timestamp of '05:52:25', a direction of 'CLIENT → SER...', and a duration of '48ms'. The message is labeled 'TOOLS/CALL' and 'de'. It includes a 'Parameters' section with the following JSON:

```
1 {  
2   "name": "detail",  
3   "arguments": {  
4     "recall_id": 0  
5   },  
6   "_meta": {  
7     "progressToken": 9  
8   }  
9 }
```

Below the parameters is a 'Response' section with the following JSON:

```
1 {  
2   "jsonrpc": "2.0",  
3   "id": 9,  
4   "result": {  
5     "content": [  
6       {  
7         "type": "text".
```

Actual invalid detail call. This crop changes only framing, not submitted inputs or returned output.

25 Domain MCP contract detail

Expected JSON schema, valid input/output, rejected JSON, complete error, and rejection reason are shown together.

DOMAIN MCP CONTRACT 2/3: detail

EXPECTED JSON INPUT SCHEMA

```
{("type":"object","additionalProperties":false,"required":["recall_id"],"properties":{"recall_id":{"type":"integer","exclusiveMinimum":0}})}
```

VALID INPUT

```
{
  "recall_id": 5505
}
```

VALID OUTPUT

```
{
  "ok": true,
  "data": {
    "id": 5505,
    "recall_code": "REC-3539-00001",
    "product_name": "Frozen Shrimp Lot 00001",
    "brand_name": "Grocery Safety Manufacturer 10",
    "category": "Produce",
    "units_affected": 6535,
    "recall_details": "Grocery Safety Manufacturer 10 Frozen Shrimp lot 00001 may contain an undeclared allergen and should be returned for a refund. Consumers should follow the recall instructions.",
    "manufacturer": {
      "id": 24,
      "name": "Grocery Safety Manufacturer 10",
      "contact_name": "Safety Contact 10"
    }
  },
  "error": null
}
```

REJECTED JSON OBJECT

```
{
  "recall_id": 0
}
```

COMPLETE RETURNED ERROR

```
{
  "ok": false,
  "data": null,
  "error": "Invalid detail input: recall_id: Input should be greater than 0"
}
```

WHY REJECTED

Database IDs begin at 1, so zero is outside the accepted domain.

The detail contract returns the stable {ok, data, error} envelope for both valid and invalid calls.

26 Actual Domain Inspector aggregate valid

A full-page close crop enlarges the actual result and Protocol input panes. Original screenshots are retained; the following contract page supplies untruncated JSON.

Results

```
1 {
2   "ok": true,
3   "data": {
4     "group_by": "category",
5     "groups": [
6       {
7         "label": "Dairy and Refrigerated",
8         "recall_count": 127,
9         "total_units": 1198565
10      },
11      {
12        "label": "Meat and Seafood",
13        "recall_count": 130,
14        "total_units": 1327145
15      },
16      {
17        "label": "Packaged Foods",
18        "recall_count": 131,
19        "total_units": 1389038
20      },
21      {
22        "label": "Produce",
23        "recall_count": 112,
24        "total_units": 1098442
25      }
26    ]
27  },
28   "error": null
29 }
```

Structured Output

```
1 {
2   "ok": true,
3   "data": {
```

Messages **LEGACY**

Newest First **Clear**

Export

05:52:53 **CLIENT -> SER...** 47ms **collapse**

TOOLS/CALL ag ↺ ⌵ ⌶ ⌵

Parameters:

```
1 {
2   "name": "aggregate",
3   "arguments": {
4     "min_units": 0,
5     "group_by": "category"
6   },
7   "_meta": {
8     "progressToken": 10
9   }
10 }
```

Response:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 10,
4   "result": {
5     "content": [
6       {
```

Actual valid aggregate call. This crop changes only framing, not submitted inputs or returned output.

26 Actual Domain Inspector aggregate invalid

A full-page close crop enlarges the actual result and Protocol input panes. Original screenshots are retained; the following contract page supplies untruncated JSON.

The screenshot displays the Domain Inspector interface, showing an invalid aggregate call. The interface is split into two main panes.

Left Pane (Results):

- Results:** A JSON object indicating an error:

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "Invalid aggregate input: group_t
5 }
```
- Structured Output:** A JSON object indicating an error:

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "Invalid aggregate input:
5 }
```

Right Pane (Messages):

- Protocol:** The selected tab.
- Console:** A search bar.
- Messages:** A list of messages. The selected message is:

```
05:53:10 CLIENT → SER... 43ms collapse
```
- Export:** A button to export the message.
- Parameters:** A JSON object representing the call parameters:

```
1 {
2   "name": "aggregate",
3   "arguments": {
4     "min_units": 0,
5     "group_by": "submitter"
6   },
7   "_meta": {
8     "progressToken": 11
9   }
10 }
```
- Response:** A JSON object representing the response:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 11,
4   "result": {
5     "content": [
6     ]
7   }
8 }
```

Actual invalid aggregate call. This crop changes only framing, not submitted inputs or returned output.

27 Domain MCP contract aggregate

Expected JSON schema, valid input/output, rejected JSON, complete error, and rejection reason are shown together.

DOMAIN MCP CONTRACT 3/3: aggregate

EXPECTED JSON INPUT SCHEMA

```
{
  "type": "object",
  "additionalProperties": false,
  "required": ["group_by"],
  "properties": {
    "group_by": {
      "type": "string",
      "pattern": "^(category|manufacturer)$",
      "min_units": {
        "type": "integer",
        "minimum": 0,
        "maximum": 1000000,
        "default": 0
      }
    }
  }
}
```

VALID INPUT

```
{
  "group_by": "category",
  "min_units": 0
}
```

VALID OUTPUT

```
{
  "ok": true,
  "data": {
    "group_by": "category",
    "groups": [
      {
        "label": "Dairy and Refrigerated",
        "recall_count": 127,
        "total_units": 1198565
      },
      {
        "label": "Meat and Seafood",
        "recall_count": 130,
        "total_units": 1327145
      },
      {
        "label": "Packaged Foods",
        "recall_count": 131,
        "total_units": 1389038
      },
      {
        "label": "Produce",
        "recall_count": 112,
        "total_units": 1098442
      }
    ]
  },
  "error": null
}
```

REJECTED JSON OBJECT

```
{
  "group_by": "submitter",
  "min_units": 0
}
```

COMPLETE RETURNED ERROR

```
{
  "ok": false,
  "data": null,
  "error": "Invalid aggregate input: group_by: String should match pattern \"^(category|manufacturer)$\""
}
```

WHY REJECTED

submitter is not an approved grouping dimension and could expose personal contact data.

The aggregate contract returns the stable {ok, data, error} envelope for both valid and invalid calls.

Part 3 Operation timeout retries and stress

The operation is executed through `Future.result(timeout=remaining)`, so a hanging call is interrupted from the caller's perspective rather than merely checked after failure.

RETRY IMPLEMENTATION AND MEASURED OUTPUT

ACTUAL OPERATION TIMEOUT

```
remaining = policy.timeout_seconds - (clock() - started)
executor = ThreadPoolExecutor(max_workers=1)
future = executor.submit(operation)
try:
    data = future.result(timeout=remaining)
except FutureTimeoutError:
    future.cancel()
    executor.shutdown(wait=False, cancel_futures=True)
    last_error = TimeoutError('operation timed out')
    break
```

POLICY

```
max_attempts=3; timeout=100ms; delays=1ms, 2ms; cap=4ms; VERIFY_SEED=263539
```

RUNTIME ASSERTION

A deliberately sleeping 200ms operation is called with a 30ms deadline. The test asserts `ok=false`, `error` contains 'timed out', and caller elapsed time is under 120ms.

FAULT OUTPUT

```
150 total calls (50 each at 0%, 20%, 50%)
Observed: immediate success; failure|success; failure|failure|failure
```

The test uses a 30 ms timeout budget and verifies that the caller returns in under 120 ms despite a 200 ms operation.

28A Actual bounded exponential backoff calculation

Exact source excerpt: code/hw5_retry.py. The measured result is immediately below.

```
delay = min(
    policy.base_delay_seconds * (2 ** (attempt - 1)),
    policy.max_delay_seconds,
)
remaining = policy.timeout_seconds - elapsed
if remaining <= 0:
    break
sleep(min(delay, remaining))
```

Verified output

```
Controlled failures before success: 2
Attempts: 3
Success: True
Requested backoff delays (ms): [1.0, 2.0]
```

29 Fault injection metrics and representative rows

The experiment used VERIFY_SEED 263539 and exactly 150 calls: 50 each at 0%, 20%, and 50% injected failure.

Injected failure	Calls	Success	Mean latency	p99 latency
0%	50	100%	0.031 ms	0.071 ms
20%	50	98%	0.323 ms	3.982 ms
50%	50	92%	0.984 ms	4.026 ms

Policy: at most three attempts, 100 ms total timeout, and bounded exponential delays of 1 ms then 2 ms (4 ms cap). p99 uses nearest rank (the maximum of 50 samples). Normal calls have negligible delay. At 50% injected failure, 92% still completed, while the remaining failures returned a clean error after exhausting the fixed attempt budget. A batch workload could tolerate more attempts and jitter; the interactive assistant favors a short, predictable response ceiling.

The numbers in this table are from the final 150-call run in raw/fault_injection_calls.csv and its matching raw/fault_injection_summary.json. Representative rows in the CSV include immediate success, failure-then-success, and three exhausted attempts.

Three measured retry outcomes from the same run

First attempt success

```
seed=263539; failure_rate=0.0; call=1
success=True; attempts=1; latency_ms=0.071
outcomes=success
error=null
```

Failure followed by retry success

```
seed=263539; failure_rate=0.2; call=13
success=True; attempts=2; latency_ms=1.4
outcomes=failure|success
error=null
```

All allowed attempts exhausted

```
seed=263539; failure_rate=0.2; call=46
success=False; attempts=3; latency_ms=3.941
outcomes=failure|failure|failure
error=operation failed after 3 attempts: injected transient storage failure
```

Part 4 execute_tool and offline assertions

Implementation and representative assertions are immediately followed by the full named PASS output.

SAFE execute_tool + OFFLINE ASSERTIONS

execute_tool CODE

```
if name == 'search' and isinstance(inputs, dict):
    limit = inputs.get('limit', 5)
    if isinstance(limit, int) and limit > 10:
        return json.dumps(envelope(error='Safety rule blocked search: limit cannot exceed 10 records'))
    return json.dumps(call_domain_tool(name, inputs, repository), sort_keys=True)
```

ASSERTION EXAMPLES

```
assert result['ok'] and result['data']['count'] == 1
assert not invalid['ok'] and 'Invalid search input' in invalid['error']
assert not blocked['ok'] and blocked['error'].startswith('Safety rule blocked')
assert run.stop_reason == 'max_steps' and run.step_count == 3
```

OUTPUT

```
PASS search valid/invalid; detail valid/invalid; aggregate valid/invalid
PASS safety allowed/blocked; PASS agent max_steps
SUMMARY 9/9 tests passed
```

Safe execution entry point, assertion examples, and 9/9 summary.

```
<      clear; .venv/bin/python scripts/test_hw05_tools.py
Vedant Vartak - DATA 260 Homework 5 offline tests
PASS test_search_valid
PASS test_search_invalid
PASS test_detail_valid
PASS test_detail_invalid
PASS test_aggregate_valid
PASS test_aggregate_invalid
PASS test_safety_rule_allowed
PASS test_safety_rule_blocked
PASS test_agent_max_steps
SUMMARY 9/9 tests passed
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

Unaltered offline-test region cropped from the original terminal capture; name, all nine PASS results and summary remain visible.
The latest 9/9 rerun is recorded in RUN_LOG.txt.

Part 5 Bounded agent loop and Ollama scenarios

The agent loop code and corresponding four-scenario terminal output are shown together.

BOUNDED AGENT LOOP CODE AND OUTPUT

CODE

```
for step in range(1, max_steps + 1):
    action = active_model.next_action(messages)
    if 'final' in action: stop_reason = 'normal_completion'; break
    result = execute_tool(action['tool'], action['inputs'], repository)
    if not parsed_result['ok'] and error.startswith('Safety rule blocked'):
        stop_reason = 'safety_rule_block'; break
    else:
        stop_reason = 'max_steps'
```

OUTPUT

```
Scenario 1: steps=2, tool_calls=1, normal_completion
Scenario 2: steps=2, tool_calls=1, normal_completion
Scenario 3: steps=2, tool_calls=1, normal_completion
Scenario 4: steps=1, tool_calls=1, safety_rule_block
```

MAX_STEPS loop, execute_tool call, normal completion, and deterministic safety stop.

```
Vedant Vartak - DATA 260 Homework 5 agent scenarios
MODEL=qwen3:8b MAX_STEPS=6 SCENARIOS=4
Scenario 1 | steps 2 | tool_calls 1 | normal_completion
    Three shrimp recalls were found: 1. **Grocery Safety Manufacturer 10** (Produce) - Frozen Shrimp Lot 00001 (REC-3539-00001), 6,535 units affected. 2. **Grocery Safety Manufacturer 09** (Dairy and Refrigerated) - Frozen Shrimp Lot 00004 (REC-3539-00004), 13,996 units affected. 3.
Scenario 2 | steps 2 | tool_calls 1 | normal_completion
    {'product': 'Frozen Shrimp Lot 00001', 'manufacturer': 'Grocery Safety Manufacturer 10 (Safety Contact 10)'}
Scenario 3 | steps 2 | tool_calls 1 | normal_completion
    Recalls grouped by category (min_units=0): Packaged Foods had the highest total units (1,389,038) and 131 recalls. Meat and Seafood followed with 130 recalls (1,327,145 units). Dairy and Refrigerated (127 recalls, 1,198,565 units) and Produce (112 recalls, 1,098,442 units) complete
Scenario 4 | steps 1 | tool_calls 1 | safety_rule_block
    Safety rule blocked search: limit cannot exceed 10 records
PASS: 3 normal completions + 1 safety-rule block
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

Three grounded completions plus one safety-rule block using qwen3:8b.

31A Complete saved agent answers

Verbatim answers from reports/hw05/raw/agent_scenario_summary.json, checked against the stop events in agent_runs.jsonl. Wrapped text supplies the continuation clipped by the terminal screenshot; these are not newly generated answers.

Scenario 1

Steps: 2 | Tool calls: 1 | Stop reason: normal_completion

Three shrimp recalls were found:

1. ****Grocery Safety Manufacturer 10**** (Produce) - Frozen Shrimp Lot 00001 (REC-3539-00001), 6,535 units affected.
2. ****Grocery Safety Manufacturer 09**** (Dairy and Refrigerated) - Frozen Shrimp Lot 00004 (REC-3539-00004), 13,996 units affected.
3. ****Grocery Safety Manufacturer 01**** (Meat and Seafood) - Frozen Shrimp Lot 00024 (REC-3539-00024), 13,039 units affected.

Scenario 2

Steps: 2 | Tool calls: 1 | Stop reason: normal_completion

{'product': 'Frozen Shrimp Lot 00001', 'manufacturer': 'Grocery Safety Manufacturer 10 (Safety Contact 10)'}

Scenario 3

Steps: 2 | Tool calls: 1 | Stop reason: normal_completion

Recalls grouped by category (min_units=0): Packaged Foods had the highest total units (1,389,038) and 131 recalls. Meat and Seafood followed with 130 recalls (1,327,145 units). Dairy and Refrigerated (127 recalls, 1,198,565 units) and Produce (112 recalls, 1,098,442 units) completed the top categories by recall count.

Scenario 4

Steps: 1 | Tool calls: 1 | Stop reason: safety_rule_block

Safety rule blocked search: limit cannot exceed 10 records

32 Agent reflection

Selected run: Scenario 4 - deterministic safety-rule block.

I selected the fourth Ollama run because it demonstrates that the harness, not the language model, owns the safety boundary. The user prompt explicitly asked the agent to retrieve recall records with a limit of 25 and told it not to reduce that limit. In step 1, local qwen3:8b produced a structured action for the search tool with query set to an empty string and limit set to 25. The harness did not send that action directly to the database. Instead, `run_agent` routed it through the single `execute_tool(name, inputs)` entry point used by every agent tool call.

Inside `execute_tool`, the domain-specific safety rule checked the requested search limit before normal schema validation or repository access. Because the limit exceeded the allowed agent maximum of 10, the function returned the shared JSON envelope with `ok=false`, `data=null`, and the error 'Safety rule blocked search: limit cannot exceed 10 records.' It raised no exception and issued no SQL query.

The loop parsed this clean result, logged the tool name, input, complete result, step number, and run identifier to `agent_runs.jsonl`, and recognized the safety-error prefix. It then stopped immediately with `stop_reason` equal to `safety_rule_block`. The run therefore used one step and one tool-call attempt, never approached the six-step ceiling, and never gave the model a chance to bypass the policy. This run shows why centralized enforcement is stronger than asking the model to follow a prompt: even when the model follows the user's unsafe request exactly, deterministic application code blocks it consistently.

Word count: 264

33 Tool contracts and AI use disclosure

Tool	Expected JSON	Rejected JSON	Complete error location
search	{"query": str, "limit": int}	{"query":"s","limit":5}	Section 23: Domain MCP contract - search
detail	{"recall_id": positive int}	{"recall_id":0}	Section 25: Domain MCP contract - detail
aggregate	{"group_by": enum, "min_units": int}	{"group_by":"submitter", "min_units":0}	Section 27: Domain MCP contract - aggregate

Sections 23, 25, and 27 contain each full expected schema, valid input/output, rejected object, untruncated returned error, and explanation. All three return {ok, data, error}. Pydantic validation is caught at the boundary, and execute_tool applies the narrower agent-only limit of 10.

AI use disclosure

1. I used an AI assistant for code explanation, implementation support, debugging, testing, and report preparation.
2. Some AI-assisted outputs needed corrections for accuracy and consistency.
3. Review feedback and comparisons with saved results helped identify those issues.
4. I requested revisions to correct the evidence and improve the final report.

34 Submission file alignment

Required item	Repository location
Final report	reports/hw05/report.pdf
Run log	reports/hw05/RUN_LOG.txt
Metrics	reports/hw05/METRICS.md
AI disclosure	reports/hw05/AI_USE.md
Verification	reports/hw05/verification.json
Raw evidence	reports/hw05/raw/
Screenshots	reports/hw05/evidence/
Postman files	reports/hw05/postman/
Canvas upload copy	output/pdf/Vartak_HW5.pdf

Temporary UI/Postman demonstration records were deleted after proof was captured; the final authenticated API run also cleaned up its temporary fixtures. The final live database therefore matches the deterministic cumulative seed.

End of report

MockModel stopping test evidence

```
def test_agent_max_steps() -> None:
    model = MockModel(
        [{"tool": "search", "inputs": {"query": "rec", "limit": 2}},
        repeat_last=True,
    )
    run = run_agent(
        "Keep searching forever",
        model=model,
        repository=REPOSITORY,
        max_steps=3,
        log_path=None,
    )
    assert run.stop_reason == "max_steps" and run.step_count == 3
```

The offline runner prints PASS test_agent_max_steps and SUMMARY 9/9 tests passed. The terminal capture below is the same retained evidence used in the report.

```
< clear; .venv/bin/python scripts/test_hw05_tools.py
Vedant Vartak - DATA 260 Homework 5 offline tests
PASS test_search_valid
PASS test_search_invalid
PASS test_detail_valid
PASS test_detail_invalid
PASS test_aggregate_valid
PASS test_aggregate_invalid
PASS test_safety_rule_allowed
PASS test_safety_rule_blocked
PASS test_agent_max_steps
SUMMARY 9/9 tests passed
vedantvartak@Vedants-MacBook-Air data260-3539 %
```